
PRACTICAL -1

bubble sort

```
import time

def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        for j in range(0, n-i-1):
            if arr[j] > arr[j+1]:
                arr[j], arr[j+1] = arr[j+1], arr[j]

n = int(input("Enter number of elements: "))

arr = []

for i in range(n):
    arr.append(int(input(f"Enter element {i+1}: ")))

start = time.perf_counter()

bubble_sort(arr)

end = time.perf_counter()

print("\nSorted Array:", arr)
print("Execution Time:", end-start, "seconds")

print("\nTime Complexity")
print("Best Case : O(n)")
print("Average Case: O(n2)")
print("Worst Case : O(n2)")
```

NAME:K.Bharath chandra
Enrolment number: (92400118611)



```
... Enter number of elements: 2
Enter element 1: 43
Enter element 2: 78

Sorted Array: [43, 78]
Execution Time: 0.00012619400001767644 seconds

Time Complexity
Best Case    : O(n)
Average Case : O(n2)
Worst Case   : O(n2)
```

selection sort

```
import time

def selection_sort(arr):
    n = len(arr)

    for i in range(n):
        minimum = i

        for j in range(i+1, n):
            if arr[j] < arr[minimum]:
                minimum = j

        arr[i], arr[minimum] = arr[minimum], arr[i]
```

NAME:K.Bharath chandra
Enrolment number: (92400118611)

```
n = int(input("Enter number of elements: "))

arr = []

for i in range(n):
    arr.append(int(input(f"Enter element {i+1}: ")))

start = time.perf_counter()

selection_sort(arr)

end = time.perf_counter()

print("\nSorted Array:", arr)
print("Execution Time:", end-start, "seconds")

print("\nTime Complexity")
print("Best Case : O(n²)")
print("Average Case: O(n²)")
print("Worst Case : O(n²)")
```

```
*** Enter number of elements: 3
    Enter element 1: 927
    Enter element 2: 028
    Enter element 3: 8263

Sorted Array: [28, 927, 8263]
Execution Time: 0.00021618900007069897 seconds

Time Complexity
Best Case : O(n²)
Average Case: O(n²)
Worst Case : O(n²)
```

insertion sort

```
import time

def insertion_sort(arr):

    for i in range(1, len(arr)):

        key = arr[i]

        j = i - 1

        while j >= 0 and key < arr[j]:
            arr[j+1] = arr[j]
            j -= 1

        arr[j+1] = key

n = int(input("Enter number of elements: "))

arr = []

for i in range(n):
    arr.append(int(input(f"Enter element {i+1}: ")))

start = time.perf_counter()

insertion_sort(arr)

end = time.perf_counter()

print("\nSorted Array:", arr)
print("Execution Time:", end-start, "seconds")
```

NAME:K.Bharath chandra
Enrolment number: (92400118611)

```
print("\nTime Complexity")
print("Best Case : O(n)")
print("Average Case: O(n2)")
print("Worst Case : O(n2)")
```

```
*** Enter number of elements: 3
    Enter element 1: 8273
    Enter element 2: 9238
    Enter element 3: 0239

Sorted Array: [239, 8273, 9238]
Execution Time: 0.00011985100002220861 seconds

Time Complexity
Best Case : O(n)
Average Case: O(n2)
Worst Case : O(n2)
```

es  Terminal

merge sort

```
import time

def merge_sort(arr):

    if len(arr) > 1:

        mid = len(arr)//2

        left = arr[:mid]

        right = arr[mid:]
```

NAME:K.Bharath chandra
Enrolment number: (92400118611)

```
merge_sort(left)
merge_sort(right)

i = j = k = 0

while i < len(left) and j < len(right):

    if left[i] < right[j]:
        arr[k] = left[i]
        i += 1
    else:
        arr[k] = right[j]
        j += 1

    k += 1

while i < len(left):
    arr[k] = left[i]
    i += 1
    k += 1

while j < len(right):
    arr[k] = right[j]
    j += 1
    k += 1

n = int(input("Enter number of elements: "))

arr = []

for i in range(n):
    arr.append(int(input(f"Enter element {i+1}: ")))

start = time.perf_counter()

merge_sort(arr)

end = time.perf_counter()
```

```
print("\nSorted Array:", arr)
print("Execution Time:", end-start, "seconds")
```

```
print("\nTime Complexity")
print("Best Case : O(n log n)")
print("Average Case: O(n log n)")
print("Worst Case : O(n log n)")
```

```
... Enter number of elements: 3
Enter element 1: 2813
Enter element 2: 9823
Enter element 3: 1092

Sorted Array: [1092, 2813, 9823]
Execution Time: 0.00012278799999876355 seconds

Time Complexity
Best Case : O(n log n)
Average Case: O(n log n)
Worst Case : O(n log n)
```

quick sort

```
import time
```

```
def quick_sort(arr):
```

```
    if len(arr) <= 1:
        return arr
```

```
    pivot = arr[len(arr)//2]
```

```
    left = [x for x in arr if x < pivot]
```

```
    middle = [x for x in arr if x == pivot]
```

```
    right = [x for x in arr if x > pivot]
```

NAME:K.Bharath chandra
Enrolment number: (92400118611)

```
return quick_sort(left) + middle + quick_sort(right)

n = int(input("Enter number of elements: "))

arr = []

for i in range(n):
    arr.append(int(input(f"Enter element {i+1}: ")))

start = time.perf_counter()

sorted_arr = quick_sort(arr)

end = time.perf_counter()

print("\nSorted Array:", sorted_arr)
print("Execution Time:", end-start, "seconds")

print("\nTime Complexity")
print("Best Case : O(n log n)")
print("Average Case: O(n log n)")
print("Worst Case : O(n2)")
```

Time Complexity Summary

Sorting Algorithm	Best Case	Average Case	Worst Case
Bubble Sort	O(n)	O(n ²)	O(n ²)
Selection Sort	O(n ²)	O(n ²)	O(n ²)
Insertion Sort	O(n)	O(n ²)	O(n ²)
Merge Sort	O(n log n)	O(n log n)	O(n log n)
Quick Sort	O(n log n)	O(n log n)	O(n ²)
Heap Sort	O(n log n)	O(n log n)	O(n log n)



Marwadi
University
Marwadi Chandarana Group



Faculty of Engineering and Technology
Department Of AI, ML & DS
Design & Analysis of algorithms(01AI0506)

```
... Enter number of elements: 3
Enter element 1: 7653
Enter element 2: 8236
Enter element 3: 1209

Sorted Array: [1209, 7653, 8236]
Execution Time: 0.0001577030 seconds

Time Complexity
Best Case      :  $O(n \log n)$ 
Average Case   :  $O(n \log n)$ 
Worst Case     :  $O(n^2)$ 
```

NAME:K.Bharath chandra
Enrolment number: (92400118611)